Object Oriented Programming and Design Methodology – CS/CE 224/272

Nadia Nasir

Habib University, Fall 2025

Templates in C++

- Templates are very useful when implementing generic (i.e. independent of the data type) constructs like vectors, stacks, lists, queues which can be used with any arbitrary type.

- The compiler generates new classes or functions when you supply arguments for these parameters; this process is called *template instantiation*

- A version of a template for a particular template argument is called a *specialization*.

- C++ provides two kinds of templates: class templates and function templates.

Function Templates

- A *function template* defines how a group of functions can be generated.

- The clue that you should create a function template is, as you might suspect, if you find you're creating a number of functions that look identical except that they are dealing with different types

- Based on the argument types provided in calls to the function, the compiler automatically instantiates separate object code functions to handle each type of call appropriately.

Syntax

- T is the template argument that accepts different data types and class is a keyword.
- You can also use class instead of typename to define a type parameter.
- The type parameter represents an arbitrary type that is specified by the caller when the caller calls the function.

```
template<typename T>
T function_name(T args)
{
……   //function body
}
```

- Normally, templates aren't compiled into single entities that can handle any type. Instead, different entities are generated from the template for every type for which the template is used.

- The process of replacing template parameters by concrete types is called instantiation. It results in an instance of a template.

```cpp
template <typename T>
T myMax(T x, T y)
{
    return (x > y)? x: y;
}

int main()
{
   cout << myMax<int>(3, 7) << endl;
   cout << myMax<char>('g', 'e') << endl;
   return 0;
}
```

Compiler internally generates and adds below code

```cpp
int myMax(int x, int y)
{
    return (x > y)? x: y;
}
```

Compiler internally generates and adds below code.

```cpp
char myMax(char x, char y)
{
    return (x > y)? x: y;
}
```

- Function templates have two kinds of parameters:
- Template parameters,
  - which are declared in angle brackets before the function template name: template <typename T> // T is template parameter .

- Call parameters,
  - which are declared in parentheses after the function template name: ... max (T const& a, T const& b) // a and b are call parameters

CLASS TEMPLATES

Class Templates

- Like function templates, class templates also provide the ability to program in a generic way.

- With class templates we can define generic purpose classes, that is, classes to which we can pass different types of data, without having to rewrite their code.

Why we need class templates?

- For example, the implementation of the stack using int*.
- The type of the elements stored in the stack is **int**.
- If we want to store some other type (e.g., strings), we have to replace all the array pointer with the new type and re-compile the program.
- That is, we cannot have two stacks in the same program, one with integer elements and the other with strings.
- We need to create a new stack (e.g., Stack_stirng), copy the code of the existing one and change the type from int to string.

- If we want to create a new stack with another data type, we have to repeat this process.

Example

```
int main ()
{

    Stack<int> mystack(4);
    mystack.push(23);
    mystack.push(55);
    mystack.push(777);
    mystack.push(86);
    mystack.showStack();

    Stack<string> mystrstack(3);
    mystrstack.push("DLD");
    mystrstack.push("OOP");
    mystrstack.push("EM");
    mystrstack.showStack();

   return 0;
}
```

```
template<typename K>
class Stack
{
    public:
        int size , topIndex;
        K   *p;
    Stack(int len)
    {
        size = len;
        topIndex = -1;
        p = new K[size];
    }
    void push(K value)
    {
        //check if stack is not full
            p[++topIndex] = value;
    }
    void showStack()
    {
        for(int i = 0 ; i < size ; i++)
            cout << p[i] << " ";
    }
};
```

Class Template Instantiation

- It is important to understand that the class template is not a class definition.

- It is just an instruction for the compiler on how to define the class.
- The class is defined when a class instantiation is required to create an object.

- For example, with the statement:

- Stack<int> mystack(5); // It is wrong to write Stack mystack(5);

- the compiler replaces the T type with int and defines the “integer” version of the class. T

The UML and Templates

Templates (also called *parameterized classes* in the UML) are represented in class diagrams by a variation on the UML class symbol. The names of the template arguments are placed in a dotted rectangle that intrudes into the upper right corner of the class rectangle.

Figure 14.3 shows a UML class diagram for the TEMPSTAK program at the beginning of this chapter.

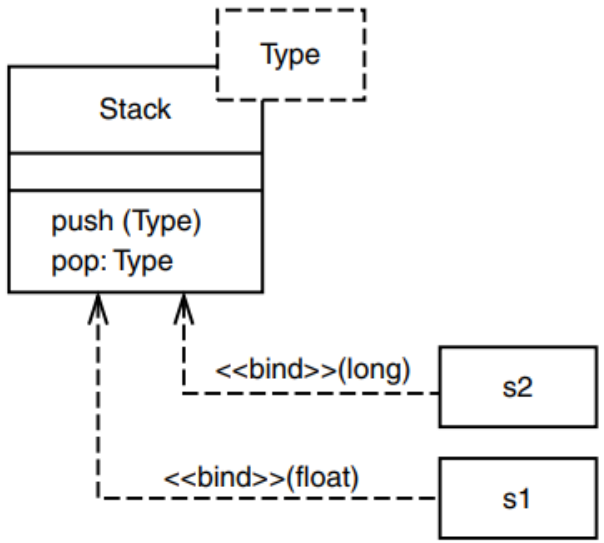

FIGURE 14.3
Template in a UML class diagram.

There's only one template argument here: `Type`. The operations `push()` and `pop()` are shown, with their return types and argument types. (Note that the return type is shown *following* the function name, separated from it by a colon.) The template argument usually shows up in the operation signatures, as `Type` does in `push()` and `pop()`.

This diagram also shows the specific classes that are instantiated from the template class: `s1` and `s2`.